Creational Design Patterns

CSE 214 — Online Exam Kit

Factory Method • Abstract Factory • Singleton • Builder

Pattern-identification techniques | Ready-to-edit Java templates
45 practice scenarios with answers | Every past question solved

Companion files: `SingletonTemplate.java`, `FactoryTemplate.java`, `AbstractFactoryTemplate.java`, `BuilderTemplate.java`

0. How to use this kit in a 20-minute exam

> **The single most important fact about this exam:** the scenario is decoration. There are only four possible answers, and each answer is a fixed skeleton of classes you already have typed out. Your entire job is (1) identify which of the four, (2) open that template, (3) rename things, (4) run it. Identification is where the marks are won or lost — not typing.

The 20-minute clock

Time	What you do	Notes
0:00 – 2:00	Read the question twice. Underline the trigger words. Decide the pattern.	Use the triage on the next page. Write the pattern name at the top of your file as a comment — it costs 5 seconds and often earns partial credit even if the code breaks.
2:00 – 4:00	Open the matching template. Save-as with the required file name.	Remember: **file name must equal the public class name**, or nothing compiles.
4:00 – 12:00	Rename classes / methods / strings to match the scenario. Delete what you don't need.	Use Find & Replace (Ctrl+H), not hand-editing. Delete extra subclasses last.
12:00 – 16:00	Write the `main()` demonstration the question explicitly asks for.	Almost every past question ends with "demonstrate / show that…". That sentence is a mark. Do exactly what it says.
16:00 – 19:00	Compile and run. Fix errors.	A compiling half-answer beats a perfect non-compiling one.
19:00 – 20:00	Final read of the question. Did you satisfy *every* bullet point?	The bullets under "Task:" are the marking scheme in disguise.

What the examiner is actually checking

1. **Does the client code avoid `new ConcreteClass()`?** (except Singleton, where the client can't use `new` at all). This one line of reasoning is the heart of all three factory-family patterns.
2. **Are the declared types interfaces / abstract types?** `Transport t = factory.create("Road");` not `Truck t = ...`.
3. **Is the required structural piece present?** private constructor (Singleton), abstract factory method (Factory Method), one factory per family (Abstract Factory), a Director (Builder).
4. **Does `main()` demonstrate the specific thing the question asked to demonstrate?**

> **Observed pattern of the paper (Jan 2026):** section A1 = Abstract Factory (Light/Dark theme kit), section C1 = Singleton (AuditLogger). Previous years show the same habit of giving different patterns to different sections, and the time dropped from 25–30 minutes to 20 minutes. That makes **Factory** and **Builder** the statistically likely candidates for the remaining section — but treat that as a hint for revision order, never as a reason to skip a pattern.

1. Identification: how to name the pattern in 90 seconds

1.1 The decision tree (run it in this order)

```
Q1. Does the question care about HOW MANY objects exist?
    ("only one", "single shared", "must not be duplicated", "same instance")
        YES -> SINGLETON. Stop.
        NO  -> Q2

Q2. Does the client end up holding ONE object that was ASSEMBLED FROM PARTS,
    in steps? ("consists of three components", "step by step", "separate
    construction from representation", "a class that directs the process")
        YES -> BUILDER. Stop.
        NO  -> Q3

Q3. Does the client need SEVERAL DIFFERENT KINDS of object that must MATCH
    each other, chosen as a set? ("family", "theme", "variant", "must not
    mix", "each theme contains three related components")
        YES -> ABSTRACT FACTORY. Stop.
        NO  -> Q4

Q4. Does the client pass a TYPE CODE and get back ONE object whose concrete
    class it must not know? ("client provides the mode as a string", "system
    decides which class to instantiate", "adding a new type later")
        YES -> FACTORY METHOD / SIMPLE FACTORY. Stop.

If two branches both fire, see the Confusion Clinic (section 3).
```

1.2 Trigger-word table (scan the question for these)

Pattern	Words that almost guarantee it
Singleton	only one instance • single shared • not more than one • global access point • strictly forbidden to have multiple copies • constructor must not be directly accessible • load once, keep in memory • same instance every time • prevent duplicates • two different modules must access the same object • audit trail in a single file
Factory Method / Simple Factory	the client provides the type as a string • decide which class to instantiate • the client should not know the concrete class names • adding a new type in future should need minimal changes • return the appropriate object • all X implement a common interface Y with a method z() • without modifying the client code
Abstract Factory	family of related products • theme / skin / style / variant / platform • must not mix components • components must match • each theme contains three related components • receives the appropriate family • change the entire family at once • cross-platform look and feel
Builder	step by step • consists of three main components • construction process involves multiple steps • separate the construction from its representation • different representations using the same construction process • a class that directs the construction • the client only asks for a model and gets the assembled product • too many constructor parameters / optional parts

1.3 Three fast structural tests

Test	Ask yourself	Verdict
The Grid Test	Can I draw a table where the columns are product *types* (Button, TextField, Dialog) and the rows are *variants* (Light, Dark)? Are BOTH dimensions > 1?	Both > 1 → **Abstract Factory**. Only rows → **Factory**.
The Output Test	At the end, does the client hold ONE object, or a SET of separate objects?	One assembled object → **Builder**. Several matching objects → **Abstract Factory**. One object, chosen by type → **Factory**.
The Parts Test	Are the "three components" *parts of one thing* (a flight, a hotel and an activity make up ONE package) or *three separate things used side by side* (a button, a text field and a dialog are three independent widgets)?	Parts of one thing → **Builder**. Separate but matching → **Abstract Factory**.

> **The Parts Test is the one that saves you.** Both Builder and Abstract Factory questions say "three components". Ask: *can I sell the button separately?* Yes → family → Abstract Factory. *Can I sell the hotel separately from the package?* The question treats the package as the deliverable → Builder.

2. The four pattern cards

CARD 1 — SINGLETON (template: SingletonTemplate.java)

Intent. Ensure a class has only one instance and provide a global point of access to it.

The three mandatory parts

```
class Logger {
    private static Logger instance;          // 1. static field holds THE object

    private Logger() {                       // 2. PRIVATE constructor
        // expensive setup happens once
    }

    public static Logger getInstance() {     // 3. global access point
        if (instance == null) {              //    lazy: create on first request
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) { System.out.println("[LOG] " + message); }
}
```

The demonstration they always ask for

```
class ModuleA { void run() { Logger.getInstance().log("A did something"); } }
class ModuleB { void run() { Logger.getInstance().log("B did something"); } }

// in main:
new ModuleA().run();
new ModuleB().run();
Logger l1 = Logger.getInstance();
Logger l2 = Logger.getInstance();
System.out.println(l1 == l2);          // true  <- THIS LINE IS THE PROOF
System.out.println(l1.hashCode() + " " + l2.hashCode());
```

Variants — and when they are actually needed

Variant	Code	Use when
Lazy (default)	`if (instance == null) instance = new X();`	Always, unless the question mentions threads. This is the expected answer.
Eager	`private static final X INSTANCE = new X();`	You want thread safety with zero extra code and don't care about lazy loading.
Double-checked locking	`private static volatile X instance;` + `synchronized` block + second null check	The question explicitly says multiple threads may call it concurrently.
Enum	`enum X { INSTANCE; }`	Shortest bullet-proof form. Safe to mention as a one-line comment; write the classic form as the main answer.

Singleton mistakes that cost marks

- Constructor left `public` or default (package-private). It must be `private`.
- `getInstance()` not declared `static` — then you'd need an object to get the object.
- The `instance` field not declared `static`.
- Calling `new Logger()` somewhere in `main()` "just to test". It won't compile, and if you made the constructor public to fix that, you deleted the pattern.
- Writing threads and `synchronized` when the paper says "thread synchronization is not required". You lose time, not marks — but time is the scarce resource.
- Forgetting the `l1 == l2` proof line when the question says "demonstrate / show that...".

CARD 2 — FACTORY METHOD & SIMPLE FACTORY (template: FactoryTemplate.java)

Intent. Define an interface for creating an object, but let a separate class or a subclass decide which concrete class to instantiate. The client works only with the product interface.

Two forms — know the difference, it is the most common viva question.
Simple Factory (not a GoF pattern, but universally accepted in courses): ONE factory class with a `static create(String type)` and a `switch`. The type code selects the class.
Factory Method (real GoF): an abstract **Creator** class declares an abstract factory method; each **subclass** overrides it to return a different product. The subclass, not a switch, makes the choice.

Most CSE 214 questions are worded as Simple Factory ("client passes a string") but sit under the heading "Factory Method". **Safest answer: write both.** They are both in the template already, so it costs you nothing.

Simple Factory skeleton

```
interface Transport { void deliver(); }

class Truck implements Transport { public void deliver() { System.out.println("By road"); } }
class Ship  implements Transport { public void deliver() { System.out.println("By sea");  } }

class TransportFactory {
    public static Transport create(String type) {          // returns the INTERFACE
        switch (type.toLowerCase()) {
            case "road": return new Truck();
            case "sea":  return new Ship();
            default: throw new IllegalArgumentException("Unknown type: " + type);
        }
    }
}
// client:  Transport t = TransportFactory.create("Road");  t.deliver();
```

Factory Method skeleton

```
abstract class Logistics {
    public abstract Transport createTransport();     // THE factory method

    public void planDelivery() {                     // creator's own business logic
        Transport t = createTransport();
        t.deliver();
    }
}
class RoadLogistics extends Logistics {
    public Transport createTransport() { return new Truck(); }
}
class SeaLogistics extends Logistics {
    public Transport createTransport() { return new Ship(); }
}
// client:  Logistics l = new RoadLogistics();  l.planDelivery();
```

How to answer "adding a new type should require minimal changes". Add one sentence as a comment: "*To add Airplane, create `class Airplane implements Transport` and one new case / one new Creator subclass. No existing client code changes — Open/Closed Principle.*" Then actually add a third product to prove it. Examiners love this.

Factory mistakes that cost marks

- Returning the concrete type: `public Truck create(...)`. It must return `Transport`.
- Putting the `switch` in `main()` instead of in the factory — that *is* the coupling the pattern removes.
- Declaring the variable as `Truck t = factory.create(...)` in the client.
- No `default` case for an unknown type code.
- Making the products extend an abstract class when the question said "implement a common interface Transport" — follow the question's own wording exactly.

CARD 3 — ABSTRACT FACTORY (template: AbstractFactoryTemplate.java)

Intent. Provide an interface for creating *families* of related objects without specifying their concrete classes, and guarantee that objects from one family are never mixed with another.

Draw the grid first — the code writes itself from it

	Button	TextField	Dialog
Light	LightButton	LightTextField	LightDialog
Dark	DarkButton	DarkTextField	DarkDialog

Columns → one **interface** each. Rows → one **concrete factory** each. Cells → one class each. For a 3×2 grid that is 3 interfaces + 6 classes + 1 factory interface + 2 factories + 1 client = 13 types. Sounds like a lot; it is 4 minutes of Find & Replace on the template.

Skeleton

```
// columns
interface Button    { void render(); }
interface TextField { void render(); }
interface Dialog    { void render(); }

// cells (row = Light)
class LightButton    implements Button    { public void render() { System.out.println("Light Button"); } }
class LightTextField implements TextField { public void render() { System.out.println("Light TextField"); } }
class LightDialog    implements Dialog    { public void render() { System.out.println("Light Dialog"); } }
// ... same three for Dark ...

// the abstract factory: ONE create method per COLUMN
interface GUIFactory {
    Button    createButton();
    TextField createTextField();
    Dialog    createDialog();
}

// one concrete factory per ROW - this is what makes mixing impossible
class LightThemeFactory implements GUIFactory {
    public Button    createButton()    { return new LightButton(); }
    public TextField createTextField() { return new LightTextField(); }
    public Dialog    createDialog()    { return new LightDialog(); }
}

// the client: receives a factory, never names a concrete product
class Application {
    private Button b; private TextField t; private Dialog d;
    Application(GUIFactory f) { b = f.createButton(); t = f.createTextField(); d = f.createDialog(); }
    void renderUI() { b.render(); t.render(); d.render(); }
}
```

The "client selects a theme" helper

```
class ThemeFactoryProvider {
    public static GUIFactory getFactory(String theme) {
        switch (theme.toLowerCase()) {
            case "light": return new LightThemeFactory();
            case "dark":  return new DarkThemeFactory();
            default: throw new IllegalArgumentException("Unknown theme: " + theme);
        }
    }
}
```

Abstract Factory mistakes that cost marks

- **The classic wrong answer:** writing one factory per *product* (a ButtonFactory, a TextFieldFactory). That is three Simple Factories, and it lets the client mix a Light button with a Dark dialog — exactly what the question forbade. One factory per **family**.
- Factory methods returning concrete classes instead of the product interfaces.
- The client class doing `new LightButton()` anywhere.
- Not showing the payoff in `main()`: build the UI with the Light factory, then with the Dark factory, using the *same* `Application` class. That is the "change the theme without changing the main logic" bullet.
- Forgetting one product type. If the question lists three components, all three need a create method.

CARD 4 — BUILDER (template: BuilderTemplate.java)

Intent. Separate the construction of a complex object from its representation, so that the same construction process can create different representations.

> **Read this before you write a single line.** Two standard models (Relaxation / Adventure, Commuter / Mountain Beast) do **NOT** mean two builders. They mean **two methods on the Director**. Different *builders* mean different *representations* of the output (the real product vs a text brochure vs an XML file). Getting this backwards is the single most common Builder error.

Skeleton

```
// 1. the product: one object, several parts
class HolidayPackage {
    private String flight, hotel, activity;
    public void setFlight(String f)   { flight = f; }
    public void setHotel(String h)    { hotel = h; }
    public void setActivity(String a) { activity = a; }
    public String toString() { return "[" + flight + " | " + hotel + " | " + activity + "]"; }
}

// 2. the builder interface: ONE METHOD PER STEP
interface Builder {
    void reset();
    void setFlight(String f);
    void setHotel(String h);
    void setActivity(String a);
}

// 3. concrete builder: assembles the real product
class HolidayPackageBuilder implements Builder {
    private HolidayPackage pkg;
    public HolidayPackageBuilder() { reset(); }
    public void reset()                  { pkg = new HolidayPackage(); }
    public void setFlight(String f)      { pkg.setFlight(f); }
    public void setHotel(String h)       { pkg.setHotel(h); }
    public void setActivity(String a)    { pkg.setActivity(a); }
    public HolidayPackage getResult()    { HolidayPackage r = pkg; reset(); return r; }
}

// 4. the DIRECTOR: one method per standard model. Knows the recipe, not the parts.
class Director {
    public void constructRelaxationPackage(Builder b) {
        b.reset();
        b.setFlight("Business Class Flight");
        b.setHotel("5-Star Resort");
        b.setActivity("Spa Treatment");
    }
    public void constructAdventurePackage(Builder b) {
        b.reset();
        b.setFlight("Economy Flight");
        b.setHotel("Mountain Cabin");
        b.setActivity("Hiking Tour");
    }
}

// 5. client
Director d = new Director();
HolidayPackageBuilder b = new HolidayPackageBuilder();
d.constructRelaxationPackage(b);
HolidayPackage p = b.getResult();      // result comes FROM THE BUILDER, not the director
```

Why `getResult()` is not in the Builder interface

Because different concrete builders return different types (a `HolidayPackage` vs a `String` brochure), and Java needs one fixed return type in an interface. Say this in a comment; it is a standard exam question. If all your builders return the same type, putting `getResult()` in the interface is perfectly acceptable — just be ready to justify it.

The fluent variant (know it, but read the question)

```
Bicycle bike = new Bicycle.Builder()
                   .frame("Carbon Fiber Frame")
                   .gears("12-Speed Gear")
                   .tires("Off-road Grip Tires")
                   .build();
```

This is the everyday Java style (static nested class, methods return `this`). Use it only when the question does **not** ask for "a class that directs the construction process". If it asks for a director, the GoF version above is what earns the marks — you can add the fluent one as a bonus.

Builder mistakes that cost marks

- One builder class per model (RelaxationBuilder, AdventureBuilder) instead of Director methods.
- No Director at all when the question says "a class that directs the construction process".
- The Director storing or returning the product. The Director only calls steps; the client fetches the result from the builder.
- The Director depending on a concrete builder type instead of the `Builder` interface — that kills the "same process, different representations" property.
- Building the product in the constructor with all three parameters. That is a telescoping constructor, i.e. the problem the pattern exists to solve.
- Forgetting `reset()`, so the second package inherits leftovers from the first.

3. Confusion clinic

3.1 Factory Method vs Abstract Factory

Factory Method	Abstract Factory
ONE product type, many implementations.	SEVERAL product types, each with many implementations.
Creator has ONE factory method.	Factory interface has ONE METHOD PER PRODUCT TYPE.
Chooses a *class*.	Chooses a *family*.
"Give me a Transport for mode 'Sea'."	"Give me the whole Dark theme kit."
Signal words: type, mode, channel, kind.	Signal words: theme, family, variant, platform, must match, must not mix.

Rule of thumb: an Abstract Factory is what you get when a class has several factory methods that all belong to the same variant. If you can honestly say "the factory has more than one create method", it is Abstract Factory.

3.2 Abstract Factory vs Builder — the hardest one

Abstract Factory	Builder
Returns products **immediately**, one call each.	Runs a **sequence of steps**, then you fetch the result at the end.
Output: several **separate** objects that happen to match.	Output: **one** object composed of the parts.
Emphasis on **consistency** ("must not mix").	Emphasis on **process** ("step by step", "multiple steps").
Varies: which **family**.	Varies: which **configuration** / representation.
Button + TextField + Dialog: three widgets you use side by side.	Frame + Gears + Tires: three parts bolted into one bicycle.

Worked disambiguation.
"*A meal combo has a Starter, a Main Course and a Drink. Two versions exist: Veg and Non-Veg. A Veg starter must never appear with a Non-Veg drink.*"
Three components + two standard versions looks exactly like the Builder questions... but the deciding sentence is **"must never appear with"**. That is a consistency constraint across a family → **Abstract Factory**.

"*A meal combo has a Starter, a Main Course and a Drink, added one after another. The kitchen has a class that knows the order of assembly for the two standard combos.*"
Here the deciding words are **"one after another"** and **"knows the order of assembly" → Builder**.

3.3 Singleton vs a class of static methods

A class with only `static` methods also gives you "one shared thing" — so why Singleton? Because a Singleton is an **object**: it can hold state, implement interfaces, be passed as a parameter, be subclassed, and be replaced with a test double. Say this in one comment line if the question hints at it.

3.4 When two patterns are both correct

Some scenarios genuinely combine patterns (a Factory that is itself a Singleton; an Abstract Factory whose concrete factories are Singletons). If you spot it, implement the pattern the question is *about*, and add one comment: `// the factory itself could be a Singleton, since it is stateless`. Never implement two patterns fully in 20 minutes.

3.5 Quick lookup: scenario noun → pattern

If the scenario is about…	…and the emphasis is…	…then
Logger, config, cache, connection pool, ID generator, spooler, session manager	uniqueness	Singleton
Transport, notification, payment method, shape, parser, codec, plugin, document exporter	choose one class at runtime	Factory
Theme, skin, OS look-and-feel, cloud provider, database vendor, furniture style, region/locale kit	matched set	Abstract Factory
Package, meal, pizza, computer, bicycle, house, CV, report, SQL query, HTTP request	assembled in steps	Builder

4. Universal mistakes (all four patterns)

Mistake	Why it costs marks / how to avoid it
File name ≠ public class name	Java refuses to compile. Decide the public class name first, save the file, then write code.
Several `public` classes in one file	Only one class per file may be `public`. In a single-file answer, make just the class holding `main` public and leave the rest package-private (no modifier). All four templates already do this.
Client code says `new ConcreteThing()`	Undoes the entire pattern. The only `new` the client should write is `new SomeFactory()` / `new SomeBuilder()` / `new Director()`.
Variables declared with concrete types	Always declare with the interface: `Transport t`, `Button b`, `GUIFactory f`.
Missing `@Override`	Not an error, but it shows intent and catches typos in method names. Cheap marks.
No demonstration in `main()`	Nearly every past paper ends with "demonstrate / show that…". Treat that sentence as a numbered requirement.
Ignoring a bullet point under "Task:"	The bullets are the marking scheme. Tick them off one by one before submitting.
Renaming half the identifiers	If the question calls it `AuditLogger`, every occurrence must say `AuditLogger`. Use Find & Replace, then compile.
Complex logic inside the product methods	Every past paper says a print statement is enough. Spending time here is spending time you need for structure.
Adding threads/synchronization unasked	Wastes minutes and adds bug surface. Only if the question says so.
No `default` / error case in a switch	One line: `throw new IllegalArgumentException("Unknown type: " + type);`
Not writing the pattern name anywhere	Put `// Pattern used: Abstract Factory` at the top. If you run out of time, that comment plus partial structure still reads as understanding.

4.1 Two-minute self-check before you submit

1. Does it compile and run? (Actually run it.)
2. Is the pattern name written as a comment at the top?
3. Singleton: private constructor + static field + static `getInstance()`? / Factory: returns the interface? / Abstract Factory: one factory per family, one create per product? / Builder: Director present, result fetched from builder?
4. Does `main()` do exactly what the last paragraph of the question asked?
5. Are all the question's class names used verbatim?

5. Every question you gave me, solved

Read these until the mapping "scenario sentence → pattern" feels automatic.

Jan 2026 — A1: cross-platform UI library, Light/Dark themes

Pattern: Abstract Factory. Decisive words: "Each theme contains three related components", "must not mix components from different themes", "receives the appropriate family", "change the complete theme without changing its main logic".
Grid: {Button, TextField, Dialog} × {Light, Dark}.
Write: 3 product interfaces; 6 concrete products; `interface GUIFactory` with `createButton/createTextField/createDialog`; `LightThemeFactory`, `DarkThemeFactory`; a client `Application(GUIFactory)`; optionally `ThemeFactoryProvider.getFactory(String)` for "the client selects a theme".
main(): render the UI with Light, then with Dark, using the same `Application` class.
Template: `AbstractFactoryTemplate.java` — delete the HighContrast block, done.

Jan 2026 — C1: online examination system, AuditLogger

Pattern: Singleton (lazy). Decisive words: "one shared AuditLogger", "only one instance can exist throughout the application", "constructor must not be directly accessible", "created when it is requested for the first time" (→ lazy, not eager), "thread synchronization is not required" (→ do *not* write `synchronized`).
Write: `AuditLogger` with private static field, private constructor, static `getInstance()` with the null check, and a `log(String)` method. Two client classes named after the question's own modules, e.g. `StudentLogin` and `ResultProcessing`.
main(): call both modules, then print `logger1 == logger2` and both hash codes.
Template: `SingletonTemplate.java` — delete Part 2 (AppConfig) and the optional variants.

Previous year — logistics: Truck / Ship, mode "Road" or "Sea"

Pattern: Factory (Factory Method). Decisive words: "all implement a common interface Transport with a method deliver()", "instantiate the correct transport object based on the requested delivery mode", "without modifying the client code", "may need to add Airplane or Train in the future".
Write: both forms from `FactoryTemplate.java`: the `TransportFactory.create(String)` switch, and the `Logistics` / `RoadLogistics` / `SeaLogistics` hierarchy. Add `Airplane` to show extensibility.

Previous year — notification library: SMS / Email / Push

Pattern: Factory. Identical shape to the logistics question: `Notification` interface with `notifyUser()`, three implementations, `NotificationFactory.create(String channel)`. The bullet "adding SlackMessage should require minimal changes" wants a comment plus, ideally, an actual fourth class.

Previous year — banking audit trail Logger

Pattern: Singleton. "Not more than one instance", "any module must access this single shared instance", "if a second instance is attempted, prevent it or return the existing one" — that last clause is literally describing `getInstance()`. The question even tells you not to bother with threads. Demonstrate from two independent clients.

Previous year — game engine GameConfig

Pattern: Singleton with state. "Loading from disk is expensive, load once and keep in memory", "strictly forbidden to have multiple copies", "no matter how many times a client tries to instantiate, they always get the same instance".
Write: the `AppConfig` half of `SingletonTemplate.java` renamed to `GameConfig`, with fields `resolution`, `audioVolume`, `difficulty`. Print a line inside the private constructor so the output proves it ran only once. Have a Graphics module and an Audio module read it.

Previous year — travel agency holiday packages

Pattern: Builder. "Consists of three main components", "the construction process involves multiple steps", "separate the construction of the complex object from its representation", "different package representations using the same construction process".
Trap: Relaxation and Adventure are **Director methods**, not two builders. The "different representations" bullet is what justifies the second concrete builder (e.g. a brochure/text builder).

Previous year — custom bicycle manufacturer

Pattern: Builder. "Step-by-step (build frame, then add gears, then add tires)", "you must create a class that directs the construction process" — an explicit order for a **Director**, so do not answer with the fluent nested-builder style alone. "The client only needs to ask for a specific model" → `director.constructCommuter(builder)` and `director.constructMountainBeast(builder)`.

6. Practice set A — name the pattern (30 rapid scenarios)

Give yourself 20 seconds each. Write the pattern *and* the phrase that decided it. Answers on the next page — cover them.

#	Scenario
1	A print spooler must have exactly one queue shared by every application on the machine.
2	A payment app supports bKash, Nagad and Card. The client passes the method name as a string and receives a `PaymentMethod` object.
3	A game renders either a Medieval world or a Sci-Fi world. Each world has a Weapon, a Character and a Building, and a Medieval weapon must never appear in a Sci-Fi world.
4	A pizza is made of base, sauce, cheese and toppings added one after another. The kitchen has two standard recipes.
5	A connection manager holds the open database connections; duplicating it would exhaust the server.
6	A report generator returns a PDF, HTML or CSV report object depending on what the user selected in a dropdown.
7	A CV generator assembles header, education, experience and skills sections; a separate class knows the layout order for the "Academic" and "Industry" formats.
8	A widget toolkit ships WindowsButton + WindowsMenu + WindowsScrollBar, and MacButton + MacMenu + MacScrollBar.
9	Application settings are read from a file at start-up and every subsystem must see the same values.
10	A delivery app must support Drone delivery next year without touching the code that triggers deliveries.
11	A meal combo has a Starter, a Main Course and a Drink. Two versions exist, Veg and Non-Veg, and items from different versions must never be served together.
12	A House object has walls, roof, and optional garage, pool and garden. The constructor has grown to nine parameters.
13	A drawing app receives "circle", "square" or "triangle" from the toolbar and produces the corresponding `Shape`.
14	An AudioManager must be unique so two background tracks can never play at once.
15	A document exporter must produce, from the same sequence of steps, either a real formatted document or a plain-text preview.
16	Every "skin" of a media player supplies a matching Slider, Window and PlayButton.
17	A rental system creates the right `Vehicle` subtype from a rental code such as "SUV" or "BIKE".
18	An in-memory cache must never be duplicated, or two parts of the app would disagree about the data.
19	A burger is assembled bun → patty → sauce → wrap; a class knows the assembly order for the "Standard Combo".
20	A data layer must work with either MySQL (MySQLConnection, MySQLCommand) or Oracle (OracleConnection, OracleCommand), never a mixture.
21	A notification service picks SMS, Email or Push at runtime from a configuration string.
22	A licence-key validator object is expensive to construct and must be identical everywhere in the app.
23	A car and its owner's manual are produced by the same assembly sequence, but one is metal and one is paper.
24	In a theming system, the Light theme must never be able to produce a Dark dialog.
25	An order-ID generator must produce strictly increasing numbers with no duplicates across the whole system.
26	A `Logistics` class has a `planDelivery()` method with real routing logic; its subclasses decide whether a Truck or a Ship is used.
27	An SQL query object is assembled from an optional WHERE, an optional ORDER BY and an optional LIMIT.
28	A furniture shop sells Modern chair + sofa + table, and Victorian chair + sofa + table.
29	A file reader returns a CSVParser, JSONParser or XMLParser based on the file extension.
30	An HTTP request object is built with an optional body, optional headers and an optional timeout before being sent.

Practice set A — answers

#	Pattern	The phrase that decided it
1	Singleton	"exactly one queue shared by every application"
2	Factory	"passes the method name as a string and receives a PaymentMethod"
3	Abstract Factory	three product types × two worlds + "must never appear in"
4	Builder	"added one after another" + two standard recipes = two director methods
5	Singleton	"duplicating it would exhaust the server"
6	Factory	one product type (report), class chosen at runtime
7	Builder	"assembles … sections" + "a separate class knows the layout order" = Director
8	Abstract Factory	3 widgets × 2 platforms, grouped by platform
9	Singleton	"every subsystem must see the same values"
10	Factory Method	"without touching the code that triggers deliveries" = Open/Closed
11	**Abstract Factory**	**Trap.** Three components looks like Builder, but "must never be served together" is a family-consistency constraint. Items are used side by side, not bolted into one object.
12	Builder	"the constructor has grown to nine parameters" = telescoping constructor
13	Factory	type code → one object
14	Singleton	"must be unique"
15	Builder	"from the same sequence of steps" → two concrete builders, one director
16	Abstract Factory	"every skin supplies a matching …"
17	Factory	rental code → subtype
18	Singleton	"never be duplicated"
19	Builder	assembly order + a class that knows it
20	Abstract Factory	two product types × two vendors, "never a mixture"
21	Factory	one product type chosen from a string
22	Singleton	"identical everywhere"
23	Builder	same steps, two representations — the textbook Builder example
24	Abstract Factory	a factory that can only produce its own family
25	Singleton	"no duplicates across the whole system"
26	Factory Method	creator with business logic; **subclasses** decide the product
27	Builder	optional parts assembled before use
28	Abstract Factory	the canonical furniture example
29	Factory	extension → parser class
30	Builder	optional parts, object finalised before being sent

Score yourself. Below 27/30, re-read section 1.3 (the three structural tests) and redo items 3, 4, 7, 11, 15, 19, 23 — those are the Builder/Abstract-Factory boundary cases that the exam actually uses to separate students.

7. Practice set B — full exam-format questions

Each of these is written in the style and length of the real paper. Do them at the keyboard, 20 minutes each, using the templates. Solutions follow.

B1 — Time: 20 minutes

You are building the checkout module of an e-commerce site. The site accepts payments through bKash, Nagad and Credit Card. All of them implement a common interface `PaymentMethod` with a method `pay(double amount)`. The checkout page sends the payment option chosen by the user as a string (e.g. "bkash"). The checkout code must not contain the names of the concrete payment classes, and the team plans to add "Rocket" next semester.

Task: Implement the design so that the checkout code receives a ready payment object for any supported option. A simple print message inside `pay()` is sufficient.

B2 — Time: 20 minutes

A university result-processing server keeps a `ResultCache` holding every published grade. Rebuilding the cache means re-reading the whole database, so it must be built once and reused. All modules (Transcript, Notice Board, SMS Notifier) must see identical data; two copies of the cache would show two different results for the same student.

Task: Implement `ResultCache` so that however many times any module asks for it, the same object is returned. Demonstrate access from two different modules and prove they hold the same object.

B3 — Time: 20 minutes

A restaurant app offers two combo styles: **Veg** ("Vegetable Soup", "Paneer Curry", "Lemon Soda") and **Non-Veg** ("Chicken Soup", "Beef Steak", "Iced Tea"). Every combo has a Starter, a Main Course and a Drink. Health regulations mean a Veg starter must never be served with a Non-Veg main course.

Task: Implement a system where the customer picks a combo style and the kitchen receives a matching set of items. Adding a Vegan style later must not change the ordering code.

B4 — Time: 20 minutes

A computer shop assembles custom PCs. A PC has a CPU, a RAM module and a Storage device. Two standard builds are sold: the **Office PC** ("Core i3", "8GB DDR4", "512GB SSD") and the **Gaming PC** ("Core i9", "32GB DDR5", "2TB NVMe"). Assembly happens in a fixed order: install CPU, then RAM, then storage.

Task: Implement the assembly so that a customer only names a standard build and receives the fully assembled `Computer`. Your design must also allow producing a printed *spec sheet* for the same build using the same assembly sequence. Use Strings for the components.

B5 — Time: 20 minutes

A media player must decode MP3, MP4 and AVI files. Each decoder implements `Codec` with a method `decode()`. The player class `MediaPlayer` contains real playback logic (`play()`) that buffers, decodes and then renders; only the choice of decoder differs between the audio player and the video player.

Task: Implement the design so that subclasses of the player, not the player itself, decide which decoder is used. Show playback for at least two player types.

B6 — Time: 20 minutes

A logistics dashboard renders in two regional presets: **Bangladesh** ("BDT Formatter", "VAT Calculator", "A4 Invoice") and **Europe** ("EUR Formatter", "GST Calculator", "A5 Invoice"). Every preset supplies a CurrencyFormatter, a TaxCalculator and an InvoicePrinter, and mixing a BDT formatter with a European invoice would produce legally invalid documents.

Task: Implement the design so that the dashboard selects a region once at start-up and every component it uses afterwards belongs to that region.

B7 — Time: 20 minutes

An IoT gateway writes every device event to one `EventJournal` file. Multiple device handlers run inside the application; if two journal objects existed, events would interleave and corrupt the file. The journal object must be created only when the first event is recorded, not at start-up, and no handler may construct it directly.

Task: Implement `EventJournal` and demonstrate that two different handlers write into the same journal. Threading is out of scope.

B8 — Time: 20 minutes

A courier company generates shipping labels. A label has a sender block, a receiver block and a service type. Two standard services exist: **Express** and **Standard**. The same generation sequence must be able to produce either a printable label object or a plain-text summary for SMS.

Task: Implement the generation process. A class must exist whose job is to run the generation steps in the right order for each service level.

Practice set B — solutions

Q	Pattern	What to write, and the decisive wording
B1	Factory (Simple Factory, optionally + Factory Method)	Decided by: "sends the payment option as a string", "must not contain the names of the concrete classes", "plans to add Rocket". Classes: `interface PaymentMethod { void pay(double amount); }`; `BkashPayment`, `NagadPayment`, `CardPayment`; `class PaymentFactory { static PaymentMethod create(String option) { switch… } }`. main: create three options in a loop, call `pay(1500)`. Add a `RocketPayment` class to prove extensibility.
B2	Singleton (lazy, with state)	Decided by: "built once and reused", "two copies would show two different results". Classes: `ResultCache` with `private static ResultCache instance`, private constructor (print "loading from database..." inside it), `public static ResultCache getInstance()`, plus `putResult`/`getResult` style methods. Two clients: `TranscriptModule`, `SmsNotifier`. main: run both modules, then `System.out.println(c1 == c2)`. The constructor's message must print once only.
B3	Abstract Factory	Decided by: three product types × two styles, "must never be served with". Classes: `interface Starter/MainCourse/Drink` (one method each, e.g. `serve()`); six concrete items; `interface ComboFactory { Starter createStarter(); MainCourse createMainCourse(); Drink createDrink(); }`; `VegComboFactory`, `NonVegComboFactory`; client `Kitchen(ComboFactory)`. "Adding a Vegan style must not change the ordering code" → one new factory class, client untouched. Say so in a comment.
B4	Builder (with Director)	Decided by: "assembly happens in a fixed order", "two standard builds", "spec sheet … using the same assembly sequence". Classes: product `Computer`; `interface ComputerBuilder { void reset(); void setCPU(String); void setRAM(String); void setStorage(String); }`; concrete `PCBuilder` (returns `Computer`) and `SpecSheetBuilder` (returns `String`); `Director` with `constructOfficePC(builder)` and `constructGamingPC(builder)`. **Trap check:** Office and Gaming are director methods; PC vs spec sheet are the two builders. Never the other way round.
B5	Factory Method (true GoF)	Decided by: "the player contains real playback logic", "subclasses, not the player itself, decide". This is the one question in the set that must *not* be answered with a switch-based factory. Classes: `interface Codec { void decode(); }`; `MP3Codec`, `MP4Codec`, `AVICodec`; `abstract class MediaPlayer { public abstract Codec createCodec(); public void play() { Codec c = createCodec(); … c.decode(); … } }`; `AudioPlayer extends MediaPlayer`, `VideoPlayer extends MediaPlayer`. main: `new AudioPlayer().play(); new VideoPlayer().play();`
B6	Abstract Factory	Decided by: "every preset supplies…", "mixing … would produce legally invalid documents", "selects a region once at start-up". Same shape as B3 with `RegionFactory`, `BangladeshFactory`, `EuropeFactory` and a `Dashboard(RegionFactory)` client.
B7	Singleton (lazy is mandatory here)	Decided by: "created only when the first event is recorded, not at start-up" → you must use the `if (instance == null)` lazy form, **not** the eager `static final` one. "No handler may construct it directly" → private constructor. Two clients: `TemperatureHandler`, `MotionHandler`. Prove with `==`.
B8	Builder (with Director)	Decided by: "the same generation sequence must produce either … or …" (→ two concrete builders) and "a class must exist whose job is to run the steps in the right order" (→ Director). Steps: `setSender`, `setReceiver`, `setServiceType`. Director methods: `generateExpressLabel(builder)`, `generateStandardLabel(builder)`. Builders: `PrintableLabelBuilder` (returns a `ShippingLabel`) and `SmsSummaryBuilder` (returns a `String`).

Three deliberate traps — make sure you can explain each

Trap 1 (B3 vs B4). Both mention three components and two standard versions. B3 says items must not be mixed and are consumed side by side → Abstract Factory. B4 says a fixed assembly order producing one `Computer` → Builder.

Trap 2 (B1 vs B5). Both are "choose an implementation". B1 hands the choice to a switch inside a factory class (Simple Factory) because the client sends a string. B5 hands the choice to subclasses of a creator that has its own logic (Factory Method). If the scenario gives the creator real work to do, it is Factory Method.

Trap 3 (B2 vs B7). Both are Singletons, but B7 explicitly forbids creating the object at start-up. Eager initialisation would be a wrong answer there, while it is acceptable in B2.

8. Scenario → template conversion table

Open the template, then run Find & Replace top to bottom in this order.

Template	Replace this	With the question's word for…
`SingletonTemplate.java`	`Logger`	the unique class (AuditLogger, GameConfig, ResultCache, EventJournal)
	`log`	its main method (record, write, notify, put)
	`ModuleA` / `ModuleB`	the two named clients (StudentLogin, ResultProcessing)
	Part 2 (`AppConfig`) + variants	delete, unless the question is about settings/state
`FactoryTemplate.java`	`Transport`	the product interface (Notification, PaymentMethod, Shape)
	`deliver`	its single method (notifyUser, pay, draw)
	`Truck`/`Ship`/`Airplane`/`Train`	the concrete products; delete spares
	`"road"`/`"sea"`…	the exact type codes from the question
`AbstractFactoryTemplate.java`	`Button`/`TextField`/`Dialog`	the three product types (Starter/MainCourse/Drink)
	`Light`/`Dark`	the two families (Veg/NonVeg, Modern/Victorian)
	`HighContrast` block	delete if only two families are asked for
	`Application`/`renderUI`	the client (Kitchen/serveCombo, Dashboard/printInvoice)
`BuilderTemplate.java`	`HolidayPackage`	the product (Computer, ShippingLabel, Bicycle)
	`setFlight`/`setHotel`/`setActivity`	the three steps (setCPU/setRAM/setStorage)
	`constructRelaxationPackage`/`constructAdventurePackage`	the two standard models
	`BrochureBuilder`	the second representation (spec sheet, SMS summary) — or delete

Compile checklist after any rename

1. Public class name = file name.
2. Every interface method implemented in every concrete class (a missing one is a compile error, not a silent bug).
3. The switch's `case` strings match what `main()` passes (watch `toLowerCase()`).
4. No leftover references to classes you deleted.

9. One-page cheat card — print this

Pattern	Fires on	Skeleton	Never forget

SINGLETON	only one • shared • global access • constructor not accessible • load once	`private static X instance;`
		`private X() {}`
		`public static X getInstance() {`
		`  if (instance == null)`
		`  instance = new X();`
		`  return instance; }`
			Prove it: `a == b` is `true`. Two client classes. Print inside the constructor so the output shows it ran once.

FACTORY	type code as a string • client must not know class names • easy to add a type	`interface P { void op(); }`
		`class A implements P {}`
		`class F {`
		`  static P create(String t) {`
		`  switch(t) { case "a": return new A(); }`
		`  } }`
			Return the **interface**. `default` case throws. Client declares `P p = F.create(…)`.

FACTORY METHOD	subclasses decide • the creator has its own business logic	`abstract class C {`
		`  abstract P createP();`
		`  void work() { createP().op(); } }`
		`class C1 extends C {`
		`  P createP() { return new A(); } }`
			The abstract method returns the product **interface**. Business logic stays in the base class.

ABSTRACT FACTORY	family • theme • variant • must match • must not mix • grid of types × variants	`interface Kit {`
		`  A createA(); B createB(); }`
		`class LightKit implements Kit {`
		`  public A createA() { return new LightA(); }`
		`  public B createB() { return new LightB(); } }`
		`class Client { Client(Kit k) {…} }`
			ONE factory per **family**, ONE create method per **product type**. Client takes the factory in its constructor and never names a concrete product.

BUILDER	step by step • three components of one object • separate construction from representation • a class that directs	`interface B { void reset();`
		`  void setX(String); void setY(String); }`
		`class PBuilder implements B {`
		`  P getResult() {…} }`
		`class Director {`
		`  void makeModel1(B b) { b.reset(); b.setX(…); } }`
			Standard models = **Director methods**. Different representations = **different builders**. Fetch the result from the **builder**.

The four sentences to say in your comments

- **Singleton:** "The constructor is private, so no other class can instantiate it; `getInstance()` creates the object on first request and returns the same reference afterwards."
- **Factory:** "The client depends only on the product interface, so a new product type needs one new class and no change to client code (Open/Closed Principle)."
- **Abstract Factory:** "Because each concrete factory can only build parts of its own family, mixing components from different families is impossible by construction."
- **Builder:** "The Director owns the sequence of steps, the Builder owns how each step is realised, so the same sequence produces different representations."

Last thing before you walk in. Open all four templates once and run them. Knowing that they compile on your machine, with your JDK, in your editor, removes the only real risk in a 20-minute exam: discovering a setup problem while the clock runs.

Structures follow the standard GoF creational patterns as presented on refactoring.guru (*Dive Into Design Patterns*); all code here is original and verified to compile and run on JDK 21.